

Lemmatization Approaches for French

Text-mining & Chatbots (Professor Thomas Gerald)

Mennaallah Khaled Abdou Salim

M2 IoT, Université Paris-Saclay

menna1salim@gmail.com

Negin Heidarifard

M2 AI, Université Paris-Saclay

heidarifardnegin@gmail.com

Maede Shabani Samgh Abadi

M2 DS, Université Paris-Saclay

maede.shabani@gmail.com

24 January 2025

Introduction

This report details our collaborative efforts to develop a **French lemmatizer** for the course *Text-mining & Chatbots* (Professor Thomas Gerald). We employed three primary strategies:

1. A **dictionary-based** approach
2. A **machine-learning** approach (via scikit-learn or PyTorch)
3. A comparison using **pretrained** tools (spaCy)

Lemmatization aims to transform a word into its canonical or base form (lemma), which is typically a real, valid word in the target language. Unlike *stemming*, lemmatization relies on linguistic knowledge, including part-of-speech (POS) information and morphological rules.

Our dataset consists of:

- `training-set.tsv`: used to train or build our dictionary/model,
- `testing-set.tsv`: used to evaluate the different approaches,
- `testing-gallica.tsv`: a gold-standard reference.

Below, we summarize the main findings for each approach.

1. Dictionary-Based Lemmatizer

Result: Dictionary-Based Lemmatizer Accuracy: **95.08%**

Methodology

- *Dictionary Construction:* We scanned each $(token, pos, lemma)$ in the training set and built a dictionary from $(token, pos)$ to its lemma.
- *Inference:* For each test token, we look up $(token, pos)$ in the dictionary; if not found, the token is used as a fallback.

Analysis

- **Strong performance on known forms** leads to a high accuracy of 95.08%.
- **Weaknesses:** The dictionary approach struggles with unseen words or ambiguous forms. Without POS, performance drops due to potential collisions (e.g., same form but different lemma).

2. Machine Learning Approach (Multinomial Naive Bayes)

Result: Naive Bayes Accuracy: $\sim 39.52\%$ (validation), $\sim 39.64\%$ (test)

Methodology

- **Feature Representation:** Used a `CountVectorizer` with a `max_features=50000` limit.
- **Classifier:** Multinomial Naive Bayes was trained to predict lemmas from the token text features.

Analysis

- **Fast but simplistic:** Naive Bayes is quick to train but fails to capture complex morphology.
- **Contextual limitation:** The bag-of-words approach offers minimal morphological detail and no sentence context.

3. SpaCy Lemmatizer

Result: SpaCy Lemmatizer Accuracy: **81.53%**

Observation

- SpaCy is a *pretrained* pipeline on large French corpora, making it robust to many standard forms.
- Accuracy of $\approx 81.53\%$ indicates good, but not top, performance on this specialized dataset.

- Domain mismatch and limited morphological coverage for specialized or archaic forms might reduce its accuracy.

4. Extended ML Approach

4.1 Traditional ML (POS vs. No POS)

We tested a scikit-learn solution to compare *with* and *without* POS:

- **Features:** token lowercasing, prefixes, suffixes, token length, capitalization checks, and optionally the POS tag.
- **Unknown Lemmas:** Introduced an “UNK” label for unseen lemmas.
- **Classifier:** A logistic-style `SGDClassifier` (`loss='log_loss'`) on top of a `DictVectorizer`.

Results:

- **With POS:** $\sim 67\% - 68\%$ accuracy
- **No POS:** $\sim 65\% - 66\%$ accuracy

Including POS yields a modest but consistent improvement (about +2% to +3%). However, the performance is still below the 95% accuracy of the specialized dictionary approach. Future improvements might include subword embeddings, better morphological features, or neural sequence models.

4.2 RNN Model (LSTM) with Word2Vec Embeddings

We also trained a recurrent neural network (LSTM) architecture that uses Word2Vec embeddings:

- **Accuracy:** 79.21%; **Validation Accuracy:** 76.87%
- **Training Details:** 15 epochs, runtime of over 1 hour.
- **Observations:** LSTMs can capture long-range dependencies and context within sequences, but performance depends heavily on hyperparameter tuning, embedding quality, and the size of the training data. Additionally, storing model parameters and Word2Vec embeddings requires significant memory.

5. Overall Comparison

6. Conclusions & Future Directions

- **Dictionary approach** achieves the highest accuracy (95%), relying on a complete and high-quality lexicon for optimal performance.
- **Naive Bayes** is a baseline classifier that underperforms in morphological tasks due to simplistic feature representation.

Approach	Accuracy	Key Points
Dictionary-Based	95.08%	High for known forms, poor for unseen
Naive Bayes (Multinomial)	39–40%	Simple model; fails with complex morphology
SpaCy (Pretrained)	81.53%	Good, but domain mismatch likely
ML (No POS) [<i>Your work</i>]	65–66%	Morphological features only
ML (With POS) [<i>Your work</i>]	67–68%	POS usage helps disambiguation
RNN (LSTM + Word2Vec)	79.21%	Long-range context, higher resource usage

Table 1: Comparison of Lemmatization Methods and Their Performance

- **SpaCy**’s pretrained pipeline highlights the potential limitations of domain mismatch, but still achieves over 80%.
- **Extended ML approaches:**
 - Traditional ML (with or without POS) can reach up to $\sim 68\%$. Improvements could come from advanced features or morphological analysis.
 - **RNN (LSTM + Word2Vec)** achieved $\sim 79\%$ accuracy, demonstrating the benefit of capturing long-range dependencies in sequences. However, it also requires more time, significant computational capacity, and hyperparameter tuning.

Choice of Method. For very fast, low-resource scenarios where high accuracy is less critical and out-of-vocabulary (OOV) words are rare, dictionary-based approaches are sufficient. Machine learning can provide a solid balance of speed, accuracy, and resource usage. Neural models such as LSTMs (or Transformers) can potentially achieve even higher accuracy when capturing context is crucial, but at a considerable cost in training time and memory.

Overall, these experiments highlight the trade-offs among dictionary-based, purely statistical, pretrained, and neural-sequence solutions. In a real-world system, the best choice may be a hybrid or a context-aware neural model that combines memorized knowledge with powerful feature extraction to maximize performance across diverse text domains.